

Intelligent Dead Reckoning

A phone that keeps a vehicle on the map when GNSS goes dark: tunnels, underpasses, multi-level parking and urban canyons. Fully offline, no server, no extra hardware.

PROBLEM STATEMENT ID

SIH26168

PROBLEM STATEMENT TITLE

AI-based dead reckoning for vehicle positioning in GNSS-denied environments

THEME

Smart Vehicles

PS CATEGORY

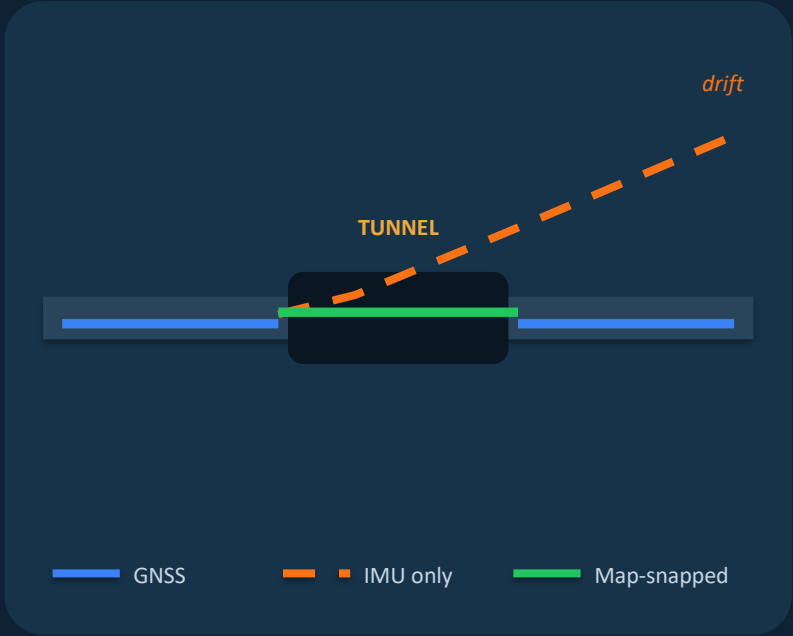
Software

TEAM ID

420

TEAM NAME

Odyssey



Three tracks drawn live on the phone. Blue is GNSS, orange is the IMU integrating alone, green is the estimate snapped to the offline road graph. The gap between orange and green is what the tunnel costs.

Idea & Proposed Solution

THE PROBLEM

- GNSS fails exactly where a driver needs it most: tunnels, underpasses, flyover stacks, parking decks and dense urban canyons.
- Consumer navigation freezes or jumps to a wrong road, and recovers only after the sky is back.
- The phone's own IMU can bridge the gap, but drifts by tens of metres within a minute if it is left to integrate on its own.

OUR SOLUTION

An Android app that fuses the phone's IMU, GNSS and an **offline OpenStreetMap road graph**, shifting trust from satellites to inertial sensing **before** the fix is lost, and snapping the drifting estimate back onto the road so the vehicle never leaves the map.

HOW IT DIFFERS

100% offline

No server, no API key

No extra hardware

Measured, not assumed

1 Strapdown inertial integrator

Attitude from the rotation vector, acceleration levelled into ENU, gravity and bias learned on the device. Zero-velocity updates pin drift at every stop.

2 GNSS trust gate

Satellite count and C/N0 collapse before fixes stop. The filter hands over to the IMU on that early evidence instead of waiting for a timeout.

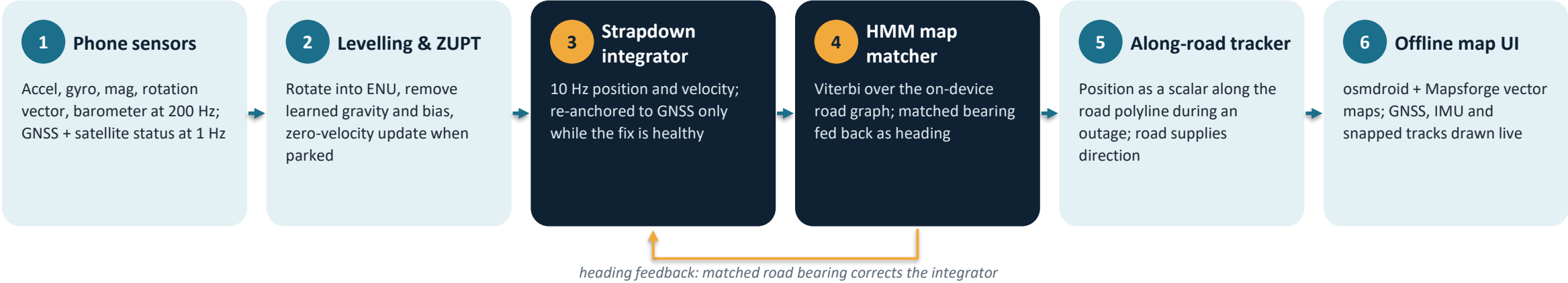
3 On-device HMM map matching

A Viterbi matcher snaps the estimate to the road and feeds the road bearing back as a heading observation. Heading error stops accumulating.

4 Road graph rebuilt on the phone

Mapsforge vector tiles carry no connectivity. We recover it by snapping endpoints on a half-metre grid, so no routing server is needed, even underground.

Technical Approach



TECHNOLOGY STACK

Mobile	Kotlin, Android foreground service, single-writer sensor thread, 2 s flush so a crash costs at most two seconds
Maps	osmdroid + Mapsforge vector .map files (OpenStreetMap data, ODbL). In-app zone downloader for all of India
AI / ML	PyTorch 1-D ResNet speed head exported to TorchScript Mobile; 40 ms per inference on a dedicated thread
Fusion	Strapdown INS + ZUPT + scalar Kalman gain on model speed; HMM Viterbi map matching; along-road tracking
Evaluation	Python harness: CRSE, CAE, AEPS on synthetic outages cut from recorded drives, scored against GNSS truth

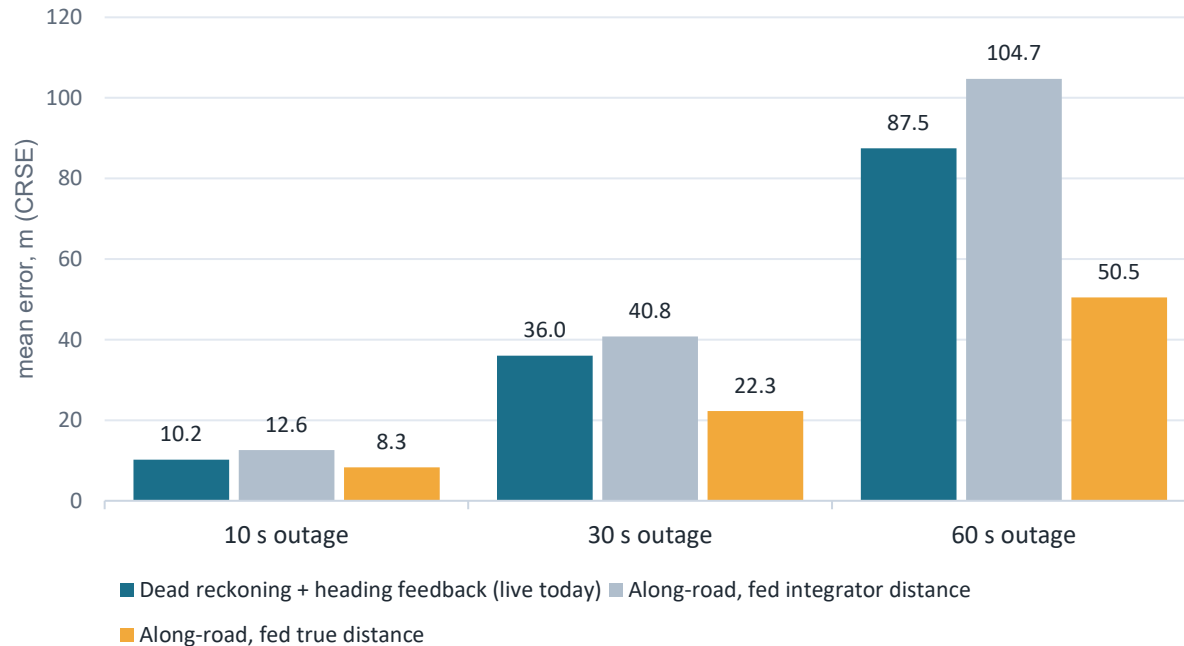
ON-DEVICE EXECUTION

Thread	Owns	Rate
imu-logger	Sensor + GNSS callbacks, all file writes, integrator	200 Hz
imu-ml	Model load and inference	10 Hz
map-match	Tile reads, HMM matcher, road graph	0.5 Hz

Exactly one writer thread, no locks. Inference results are posted back to the logger thread; stale requests are dropped rather than queued. GNSS is trusted only with 4+ satellites used and accuracy under 20 m.

Feasibility & Viability

MEASURED ON DEVICE: POSITION ERROR DURING SYNTHETIC GNSS OUTAGES



46 to 57 outages per duration, cut from recorded sessions and scored against GNSS ground truth. Along-road tracking cuts 60-second error by 42% once it is fed a trustworthy distance.

CHALLENGES & HOW WE CLOSE THEM

Along-track bias

A -0.02 m/s^2 acceleration bias costs ~30% of distance over 60 s unaided. Fix: learn horizontal bias against GNSS velocity while the fix is healthy, carry it into the outage.

Under-trained speed model

304 training windows, walking pace only. Fix: record vehicle-speed drives, retrain, and enable speed fusion only if it beats a constant predictor.

Map size and field validation

The 210 MB zone map ships as a release asset with an in-app downloader. Heading feedback has run only in replay; next step is a live road test.

WORKING TODAY

- ✓ Android app records 11 sensor streams on a Samsung Galaxy S21 FE, time-aligned on one monotonic clock
- ✓ Dead reckoning with map-matched heading feedback runs on the phone and is measured, not simulated
- ✓ Road topology recovered on device: 25,383 segments around IIT Kharagpur, 100% linked, 98.5% in one component
- ✓ Offline maps render with no tile server; zones for the rest of India download in-app

Impact & Benefits

0

cloud calls, servers or API keys

10 Hz

position updates on the handset

42%

lower 60 s outage error with along-road tracking

100%

of India coverable with free OSM zone maps

Social

- Drivers, ambulances and delivery riders keep guidance through tunnels, underpasses and hill-road cuttings
- No wrong turn at a tunnel exit: the position is already on the right road when the sky returns
- Works on the phone people already own; nothing to buy or install in the vehicle

Economic

- Zero recurring cost: no map API billing, no data plan, no server to run
- Fleet and logistics tracking that does not go blind under flyovers or in parking decks
- One codebase for OEM navigation, ride-hailing and toll or road-agency apps

Strategic & Environmental

- Location never leaves the device: privacy by construction
- Offline by design, so it serves low-connectivity and remote regions first, not last
- Fewer missed exits and detours means less fuel burnt; road topology from OSM is open and reusable

Scalability. One Mapsforge file per zone (210 MB for eastern India, 1.5 GB for the whole country). The same estimator extends to metro and rail, indoor parking and two-wheelers, and its measured outage harness lets every future change be gated on numbers.

Research & References

LITERATURE WE BUILD ON

Brossard, Barrau, Bonnabel (2019). AI-IMU Dead-Reckoning. arXiv:1904.06064

Hurwitz, Cohen, Klein (2024). Deep Learning Assisted Inertial Dead Reckoning and Fusion. arXiv:2407.16387

Gomes, Costa (2020). Vehicular Dead Reckoning Based on Machine Learning and Map Matching. IEEE

Suwandi, Kitasuka, Aritsugi (2019). Vehicle Vibration Error Compensation on IMU-accelerometer Sensor Using Adaptive Filter and Low-pass Filter Approaches. J. Inf. Processing 27

Nirmal et al. (2016). Noise modeling and analysis of an IMU-based attitude sensor. arXiv:1608.07053

Chao et al. (2022). A Comprehensive Review of Map-Matching Techniques. Int. J. Web Services Research 19(1)

Newson, Krumm (2009). Hidden Markov Map Matching Through Noise and Sparseness. ACM SIGSPATIAL

IO-VNBD (Coventry Univ.). Inertial and Odometry Vehicle Navigation Benchmark Dataset for model training and CRSE / CAE / AEPS metric definitions

EXISTING SOLUTIONS STUDIED

Google Maps SDK

No offline tile access, no dead-reckoning hook, billing account required. We use OSM data under ODbL instead.

OSRM-based map matching

Accurate, but needs a routing server. We rebuild road connectivity on the handset from Mapsforge tiles.

Learned inertial odometry (RoNIN, AI-IMU)

Strong results with dense training data. Our speed head reuses the idea but is gated until it beats a constant.

Open data and tools. OpenStreetMap (ODbL), Mapsforge map builds, osmdroid, PyTorch Mobile, Android Sensor and GNSS status APIs. Every number in this deck is reproducible from the committed evaluation harness against recorded sessions.